

Dokumentacja Projektu: GBMP (Graph-Based Music Recommender)

Konrad Ciężadło

1. Wstęp

1.1 Cel projektu

Celem projektu jest opracowanie aplikacji webowej Graph-Based Music Recommender typu SPA. System wykorzystuje grafową bazę danych Neo4j do modelowania relacji pomiędzy utworami muzycznymi, ich wykonawcami oraz przypisanymi do nich tagami (określającymi m.in. klimat, dekadę czy gatunek). Aplikacja służy do rekomendowania muzyki oraz iteracyjnej eksploracji bazy danych na podstawie cech wspólnych utworów i parametrów audio (Energy, Danceability).

1.2 Zakres projektu

System realizuje:

- Wyszukiwanie utworów (wyszukiwanie pełnotekstowe po tytule i wokaliście z mechanizmem autouzupełniania).
- Generowanie rekomendacji metodą *Content-Based Filtering* na podstawie analizy grafowej (współdzielone tagi).
- Nakładanie dodatkowych filtrów parametrycznych (odchylenie lat wydania, odchylenie parametrów audio, wymuszenie konkretnego wykonawcy).
- Iteracyjną eksplorację muzyki (wyszukiwanie za pomocą płynnie budowanej ścieżki tagów z podpowiadaniem najbardziej powiązanych kategorii).
- Integrację z zewnętrznymi serwisami (przekierowanie do serwisu Spotify za pomocą spotify_id).

System nie realizuje:

- Logowania, rejestracji ani zarządzania kontami użytkowników.
- Bezpośredniego odtwarzania plików dźwiękowych w przeglądarce (poza zewnętrznymi linkami).

1.3 Wykorzystane technologie

Warstwa	Technologia
Frontend	HTML5, CSS3, JavaScript
Backend	Python 3, Flask, Flask-CORS

Zarządzanie pakietami	uv
Baza danych	Neo4j (AuraDB)
ORM / Driver	Neo4j Python Driver

2. Analiza problemu

2.1 Opis problemu

Zarządzanie muzyką i jej rekomendowanie to wyzwanie polegające na łączeniu utworów, które z mogą się od siebie różnić, ale posiadają wspólny "klimat". Konwencjonalne, relacyjne bazy danych (SQL) świetnie radzą sobie z przypisywaniem utworów do jednego gatunku, ale zawodzą pod względem wydajności, gdy utwór posiada kilkanaście tagów, a system musi w czasie rzeczywistym odnaleźć utwory krzyżujące się w wielowymiarowej przestrzeni cech. Bazy grafowe rozwiązują problem złożoności obliczeniowej poprzez traktowanie relacji (np. HAS_TAG) jako fizycznych wskaźników.

2.2 Wymagania funkcjonalne

1. Tryb Song Recommender:

- Wyszukiwanie utworu po nazwie lub wykonawcy z opcją przełączania trybu (switch Title / Artist).
- Zmiana wartości precyzji wyszukiwania dla parametrów Energy Match (+/- %) oraz Year Match (+/- lata).
- Opcja zaznaczenia filtru Only THIS Artist.
- Wyświetlanie szczegółów wybranego utworu bazowego oraz dynamicznie generowanej listy wyników z wyliczoną w procentach wartością dopasowania (Match).

2. Tryb Discover by Tags:

- Inicjalizacja widoku najpopularniejszymi tagami w bazie.
- Dynamiczne renderowanie kolejnych, powiązanych tagów po kliknięciu.
- Wybór zakresu lat (Release Year Range).
- Generowanie docelowej listy utworów na podstawie wyselekcjonowanej kombinacji tagów.

2.3 Wymagania нефункционалне

- **Architektura SPA:** Płynne przełączanie między zakładkami oraz ładowanie list bez przeładowywania (odświeżania) okna przeglądarki.
- **Wydajność:** Realizacja zapytań grafowych w czasie rzeczywistym, pozwalająca na natychmiastową reakcję interfejsu (np. przy wpisywaniu w wyszukiwarkę).

3. Projekt grafowej bazy danych

3.1 Model grafu

Baza opiera się na prostym modelu wydzielającym obiekty fizyczne od cech w celu maksymalizacji punktów wspólnych (węzłów przecięcia).

Węzły (Nodes):

- **Track:** Przechowuje: spotify_id, name, year, danceability, energy.
- **Artist:** Przechowuje właściwość name.
- **Tag:** Przechowuje właściwość name.

Relacje (Relationships):

- **PERFORMED_BY:** relacja od utworu do artysty.
- **HAS_TAG:** relacja od utworu do wielu tagów.

3.2 Diagram grafu

```
(:Artist) <-[:PERFORMED_BY]- (:Track) -[:HAS_TAG]-> (:Tag)
```

3.3 Uzasadnienie wykorzystania grafowej bazy danych

Wyznaczanie rekomendacji wymaga wielokrotnego przechodzenia przez tabele krzyżowe (łączące utwory z dziesiątkami tagów). W bazie relacyjnej SQL znalezienie "piosenek, które współdzielał najwięcej tagów z piosenką X" wymagałoby niezwykle ciężkich sprzężeń (JOIN) i grupowania wielkich zbiorów danych, co drastycznie obciążałoby CPU i pamięć RAM. W Neo4j przechodzenie wzdłuż połączeń pomiędzy węzłami odbywa się szybko, bez względu na całkowity rozmiar bazy danych. Rozwiązuje to problem tzw. "eksplozji kartezjańskiej" w systemach rekomendacyjnych.

4. Diagramy UML

4.1 Diagram przypadków użycia

Aktor: Użytkownik przeglądarki

Przypadki użycia:

- Wyszukiwanie piosenki (wraz z podpowiedziami autouzupełniania).
- Przeglądanie szczegółów piosenki i jej wskaźnika dopasowania.
- Iteracyjne wybieranie i odznaczanie tagów muzycznych.
- Modyfikacja filtrów wyszukiwania (tolerancja roku, tolerancja energii).

4.2 Data Transfer Objects

Najważniejsze struktury danych wymieniane między Backendem a Frontend-em:

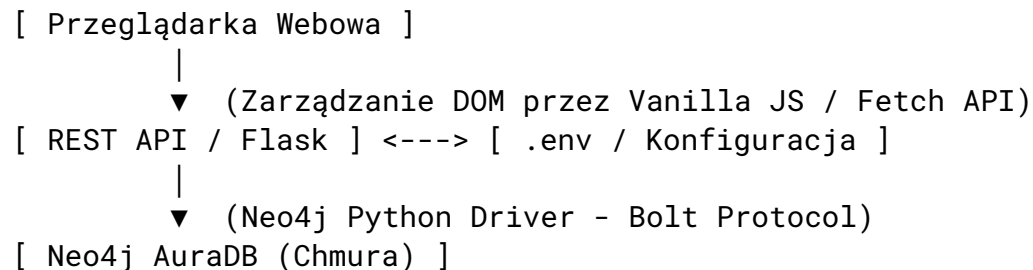
- TrackItem (spotify_id, title, artist) → Dla wyszukiwarki.
- SeedDetails (title, artist, year, tags[], spotify_id) → Dla nagłówka rekomendacji.
- RecommendedTrack (title, artist, year, match_percentage, shared_tags[], spotify_id) → Dla listy wyników.
- TagNode (tag_name, frequency) → Dla budowy przycisków eksploratora.

4.3 Schemat przepływu danych (Data Flow)

1. **Wyszukiwanie:** Użytkownik wpisuje ≥ 3 znaki → Moduł api.js (Frontend) wysyła żądanie do API → routes/tracks.py składa zapytanie CONTAINS w Cypherze → Zwraca TrackItem[].
2. **Generowanie Rekomendacji:** Użytkownik wybiera piosenkę i ustawia zakres year → Moduł api.js wysyła żądanie do API → Backend dynamicznie buduje klauzulę WHERE uwzględniając tolerancję → Neo4j przechodzi przez wierzchołki (:Tag) zliczając przecięcia → Zwracana jest lista RecommendedTrack[].

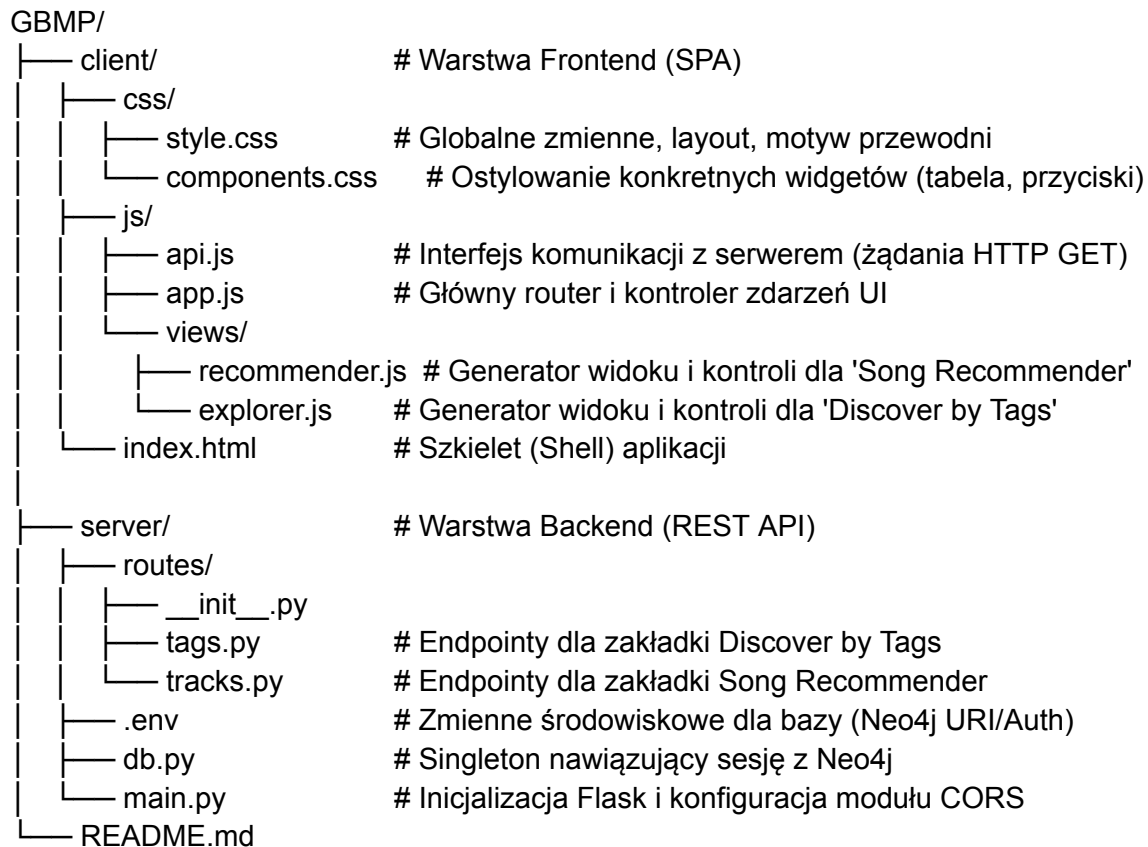
5. Architektura systemu

System funkcjonuje w architekturze klient-serwer:



6. Implementacja

6.1 Struktura projektu



6.2 Kluczowe moduły

- **Widok Song Recommender:** Przejmuje `<section id="content-top">` dla paska wyszukiwania i przełączników filtrujących. Dynamicznie renderuje `<section id="content-list">`, wyróżniając w pierwszym wierszu wybraną piosenkę (Seed), a poniżej sortując utwory docelowe z kalkulacją dopasowania $(\text{shared_tags_count} / \text{seed_tags_count}) * 100$.
- **Widok Discover by Tags:** Zastępuje interfejs blokami tagów. Na podstawie zwrotki z `/tags/related`, kontroler dobudowuje nowe przyciski do siatki, a równolegle aktualizuje tabelę piosenek poniżej suwaka lat.

7. Zapytania grafowe i endpointy (REST API)

Poniżej zestawiono kluczowe zapytania do bazy danych, połączone z endpointami wystawionymi w module Flask.

7.1 Wyszukiwarka z Autouzupełnianiem

- **Cel:** Zwrócenie szybkiej listy podpowiedzi podczas pisania (dopasowanie częściowe).
- **Endpoint:** GET /tracks/search?q={text}&search_by={title|artist}
- **Zapytanie Cypher:**

```
MATCH (t:Track)-[:PERFORMED_BY]->(a:Artist)
WHERE toLower(t.name) CONTAINS toLower($q) // Zmiana na a.name w
przypadku wyszukiwania po artyście
RETURN
    t.spotify_id AS spotify_id,
    t.name AS title,
    coalesce(a.artist, a.name) AS artist
ORDER BY t.name ASC LIMIT 10
```

7.2 Główny silnik rekomendacji

- **Cel:** Zwrócenie podobnych utworów z możliwością aplikacji modyfikatorów.
- **Endpoint:**
GET /tracks/<id>/recommendations?energy_tolerance=0.1&year_tolerance=3
&artist_match=true
- **Zapytanie Cypher (Wersja pełna z filtrami):**

```
MATCH (seed:Track {spotify_id:
$spotify_id})-[:HAS_TAG]->(seedTag:Tag)
WITH seed, count(seedTag) AS total_seed_tags
MATCH (seed)-[:HAS_TAG]->(sharedTag:Tag)-[:HAS_TAG]-(recom:Track)
MATCH (recom)-[:PERFORMED_BY]->(artist_node:Artist)
WHERE seed.spotify_id <> recom.spotify_id
    AND recom.energy >= seed.energy - $energy_tol
    AND recom.energy <= seed.energy + $energy_tol
    AND recom.year >= seed.year - $year_tol
    AND recom.year <= seed.year + $year_tol
// OPCJA ARTIST MATCH: Ograniczenie artist_node do artysty seed'a
RETURN
    recom.name AS title,
    coalesce(artist_node.artist, artist_node.name) AS artist,
    recom.year AS year,
    total_seed_tags AS seed_tags_count,
    count(sharedTag) AS shared_tags_count,
    collect(coalesce(sharedTag.tag, sharedTag.name)) AS shared_tags,
    recom.spotify_id AS spotify_id
ORDER BY shared_tags_count DESC LIMIT 10
```

7.3 Mechanizm Podpowiadania Tagów

- **Cel:** Znalezienie tagów najczęściej współwystępujących z tagami już zaznaczonymi w interfejsie.
- **Endpoint:** GET /tags/related?tags=rock,90s
- **Zapytanie Cypher:**

```
MATCH (tr:Track)
WHERE all(t IN $selected_tags WHERE exists {
    MATCH (tr)-[:HAS_TAG]->(tagNode:Tag)
    WHERE tagNode.tag = t OR tagNode.name = t
})
MATCH (tr)-[:HAS_TAG]->(relatedNode:Tag)
WITH coalesce(relatedNode.tag, relatedNode.name) AS related_tag,
$selected_tags AS selected_tags
WHERE NOT related_tag IN selected_tags
RETURN related_tag, count(*) AS frequency
ORDER BY frequency DESC LIMIT 15
```

7.4 Znajdowanie utworów dla danych Tagów

- **Cel:** Wyświetlenie piosenek podpowiadających wybranemu "vibe'owi" i zadanemu okresowi czasowemu.
- **Endpoint:** GET /tracks?tags=rock,90s&year_min=1990&year_max=1999
- **Zapytanie Cypher:**

```
MATCH (tr:Track)-[:PERFORMED_BY]->(a:Artist)
WHERE all(t IN $selected_tags WHERE exists {
    MATCH (tr)-[:HAS_TAG]->(tagNode:Tag)
    WHERE tagNode.tag = t OR tagNode.name = t
})
AND tr.year >= $year_min AND tr.year <= $year_max
RETURN
    tr.name AS title,
    coalesce(a.artist, a.name) AS artist,
    tr.year AS year,
    tr.spotify_id AS spotify_id
ORDER BY tr.year DESC LIMIT 20
```